Now available in grey

Semi-simplicial groupoids from weakenings and thinnings

Thorsten Altenkirch
University of Nottingham, United Kingdom
Thorsten.Altenkirch@nottingham.ac.uk

October 2026

Abstract

In a companion note [2] we defined a small type theory of weakenings and thinnings, in which semi-simplicial types up to level n can be constructed uniformly in n as contexts $\mathrm{SST}\, n$. The syntax is defined by induction-recursion without K, and it is a set. Its naive interpretation in the universe, however, needs uniqueness of identity proofs (UIP) for the semantic types. Here we lift this limitation by one level. We interpret the syntax in families of *groupoids*, without UIP and without K. Two changes make this work: a *Forded* semantics of telescope extension, in which a kept entry never transports its value but only changes a stored path, and a semantic action of the codes for equality of thinnings, defined by recursion. What remains are 2-dimensional coherences of the semantic composition and weakening, which we prove by induction; their 3-dimensional components are trivial because the semantic types are groupoids. As a result, $[\![\mathrm{SST}\, n]\!]$ is a type of semi-simplicial groupoids up to level n, defined uniformly in n. We also report on the next level, 2-types, where the 3-dimensional coherences have to be proved. The development is formalised in Cubical Agda, in the repository github.com/txa/sst-new.

"Any customer can have a car painted any colour that he wants so long as it is black."

— Henry Ford, *My Life and Work* [1]

From 1914 Ford's Model T was sold only in black; other colours returned in 1926. The companion note [2], whose title borrows Ford's remark, found that the naive interpretation of our syntax works in any colour, as long as it is UIP, that is, in sets. This note offers the next shade: groupoids.

1 Introduction

Semi-simplicial types are the simplest structure in homotopy type theory that needs an infinite tower of coherences, and it is a well-known open problem whether the type of semi-simplicial types up to level n can be defined uniformly in a variable $n : \mathbb{N}$ in book HoTT [11, 10]. In two-level type theory this is possible [3, 4], and Kolomatskaia and Shulman define them in displayed type theory [8].

The companion note [2] explains an attempt that proceeds in two steps. First define a small type theory (TT) whose contexts can represent the data of semi-simplicial types, and construct the contexts $\mathrm{SST}\, n$ uniformly in that syntax. Then interpret the syntax in the universe. The first step

goes through without K. The second one does not: the naive interpretation transports along the semantic images of equations that hold only propositionally, and making these transports agree needs UIP. The note also explains that the problem cannot simply be moved into the syntax: every presentation we tried either has critical pairs in the syntax or a naive model that needs UIP.

This note shows that the limitation can be lifted, at least by one level. We interpret `TT` in families of groupoids, without UIP and without K (file `experiments/sem1/Sem1.agda`). The coherences that the naive interpretation takes from UIP are now proved by induction on the syntax. They are 2-dimensional, and the only use of the truncation is that their 3-dimensional components are trivial in groupoids. As a consequence

$$[\![\mathrm{SST}\, n]\!] : \mathcal{U}_1$$

is a type of semi-simplicial groupoids up to level n, defined uniformly in n. We do not claim that this is the first such definition; truncated versions of the problem need only finitely many coherences, and are expected to be solvable. The point is that the same syntax, and the same uniform construction of $\mathrm{SST}\, n$, work, and that the semantics has a clear structure that suggests how to go further.

Sections 2–4 recall the syntax, the construction of $\mathrm{SST}\, n$ and the naive interpretation from [2]. Section 5 is the main part: the groupoid semantics. Section 6 discusses the resulting semi-simplicial groupoids, and Section 7 the next level.

2 Semi-simplicial types up to triangles

A semi-simplicial type up to level 2 consists of

$$\begin{aligned} &X_0 : \mathcal{U} \\ &X_1 : X_0 \to X_0 \to \mathcal{U} \\ &X_2 : (x_0\, x_1\, x_2 : X_0) \to X_1\, x_0\, x_1 \to X_1\, x_0\, x_2 \to X_1\, x_1\, x_2 \to \mathcal{U} \end{aligned}$$

Points, edges between points, and triangles filling a boundary of three points and three edges. Each X_k is a family over a *telescope*, and each entry of the telescope is an earlier family applied to a *selection* of earlier entries: the edge $x_{02} : X_1\, x_0\, x_2$ selects x_0 and x_2. Selections compose, and at level 2 this composition is used for the first time in a non-trivial way, since the boundary of a triangle consists of edges and the boundary of an edge consists of points.

3 The type theory of weakenings and thinnings

The syntax has five sorts (see [2, §§3–5] for a more leisurely account):

`Con : Set`	contexts	sequences of type families $X_0, X_1, \ldots$
`Ty : Con → Set`	types	telescopes (domains of families)
`Var : (Γ : Con) → Ty Γ → Set`	variables	a family in Γ over a telescope
`Wk : Con → Con → Set`	weakenings	here only `p : Wk (Γ ▷ A) Γ`
`Thin : Ty Γ → Ty Γ → Set`	thinnings	projections between telescopes

A thinning keeps some entries of a telescope and drops the others; read backwards, it is an order-preserving injection of the entries of its target into those of its source. The constructors are

```
•    : Con
_▷_  : (Γ : Con) → Ty Γ → Con          -- add a family over a telescope
ε    : Ty Γ                            -- empty telescope
ext  : (A : Ty Γ) → Var Γ C → Thin A C → Ty Γ
                                       -- A extended by an entry  x (δ a)
zero : Var (Γ ▷ A) (wkTy A p)
suc  : Var Γ A → Var (Γ ▷ B) (wkTy A p)
tε   : Thin ε ε
t-d  : Thin A B → Thin (ext A x γ) B   -- drop the last entry
t-k  : (σ : Thin A B) → CodeV x y → Code (δ ∘ σ) δ'
     → Thin (ext A x δ') (ext B y δ)   -- keep it
```

Weakening and composition are operations,

```
wkTy   : Ty Γ → Wk Δ Γ → Ty Δ
wkVar  : Var Γ A → (w : Wk Δ Γ) → Var Δ (wkTy A w)
wkThin : Thin A B → (w : Wk Δ Γ) → Thin (wkTy A w) (wkTy B w)
_∘_    : Thin B C → Thin A B → Thin A C
```

defined by recursion, simultaneously with the syntax, since they occur in the indices of `zero`, `suc` and `t-k`. So the syntax is an inductive-inductive-recursive definition. Agda accepts it, but we know of no general account of definitions of this form; we discuss this briefly in Section 8.

A kept entry stores the equation that relates its index `δ'` to the composite `δ ∘ σ` (it is *Forded*). Instead of an identity type it uses a code for equality of thinnings, defined by recursion:

```
Code : Thin A C → Thin A' C' → Set
Code tε          tε          = ⊤
Code (t-d σ)     (t-d τ)     = Code σ τ
Code (t-k σ _ _) (t-k τ _ _) = Code σ τ
Code _           _           = ⊥
```

and similarly `CodeV` for variables. Codes are propositions, by a plain induction, and they decode to equations (`Code-sound`). With them composition is defined without transport, for instance

```
t-k δ ey q ∘ t-k σ ex e = t-k (δ ∘ σ) (CodeV-trans ex ey) (k-code q e)
```

and the syntax is K-free (file `TT.agda`). Its sorts are sets, by encode-decode [7, Theorem 7.2.2].

SST in the syntax. Following [3, Section 3], we define by recursion on k, simultaneously, the context `ctx k` of levels $X_0, \dots, X_{k-1}$, the k-skeleton `SK k n : Ty (ctx k)` of an $(n-1)$-simplex, its face maps `SK→ k f : Thin (SK k n) (SK k m)` for order preserving embeddings f, and their functoriality `α k f g`, a code relating `SK→ k g ∘ SK→ k f` and `SK→ k (f ∘ g)`. This gives

```
SST : ℕ → Con
```

uniformly in n and without K (file `SSTk.agda`).

4 The naive interpretation needs UIP

The intended interpretation of a context is a nested Σ-type of families, of a telescope a type depending on the context, and of a thinning a projection:[1]

[1] We write Σ-types as `(x : A) × B`, by analogy with `(x : A) → B`, and let `×` associate to the right. This is not Agda syntax: in Agda the type is written `Σ A (λ x → B)`, or `Σ[ x ∈ A ] B` with the standard library.

```
⟦ Γ ▷ A ⟧C       = (γ : ⟦ Γ ⟧C) × (⟦ A ⟧T γ → Set)
⟦ ext A x δ ⟧T γ = (a : ⟦ A ⟧T γ) × ⟦ x ⟧V γ (⟦ δ ⟧Th a)
⟦ t-k σ _ e ⟧Th (a , v) = ⟦ σ ⟧Th a , subst ⟦ x ⟧V (keq e a) v
```

where `keq e a : ⟦ δ' ⟧Th a ≡ ⟦ δ ⟧Th (⟦ σ ⟧Th a)` is built from the interpretation of the stored code and from

```
⟦∘⟧ : ⟦ δ ∘ σ ⟧Th a ≡ ⟦ δ ⟧Th (⟦ σ ⟧Th a)
```

which holds only propositionally. In the case `t-k ∘ t-k` of ⟦∘⟧ itself, a value is transported along two different paths, and we have to show that the results agree. Since the semantic types are arbitrary, this is UIP for them [2, §7].

Note that this is a property of the naive *semantics*, not of the syntax. The syntax is a set, and that does not need UIP: it follows from the codes by encode-decode. Nor does set-ness of the syntax restrict its interpretations, since the syntax is not a quotient. What fails is the particular way in which the naive interpretation interprets a kept entry, by transporting its value along the image of a propositional equation. With arbitrary types the coherence of these transports cannot be proved, and it is what the interpretation in [2] takes from a UIP assumption (`pair-irr`, file `Sem.agda`).

5 The groupoid semantics

We now restrict the semantic families to groupoids and show that then all the coherences can be proved. The interpretation is in Cubical Agda [12], file `experiments/sem1/Sem1.agda`. For equality reasoning we use Agda's inductive identity type, for which J computes; it is equivalent to the cubical path type (`Cubical.Data.Equality`), and we use the cubical library only to state and transfer h-levels.

5.1 The interpretation

Groupoids. A family in a context is now a family of groupoids:

```
record Grpd : Set₁ where
  field ⌞_⌟ : Set ; isGrpd : isGroupoid ⌞_⌟

⟦ • ⟧C     = ⊤
⟦ Γ ▷ A ⟧C = (γ : ⟦ Γ ⟧C) × (⟦ A ⟧T γ → Grpd)
```

The interpretations of types (telescopes) are then groupoids as well (`isGrpdT`), since groupoids are closed under Σ and path types of groupoids are sets. This is all we need: the stored paths, and the coherences between them, live in the interpretations of telescopes. The interpretation of a context is not a groupoid, since it contains the universe of groupoids, which by univalence is a 2-type; but nothing is needed about its h-level.

Forded extension. The first change is to Ford the semantics of telescope extension, just as the syntax Fords the index of a kept entry:

```
⟦ ext A x δ ⟧T γ = (a : ⟦ A ⟧T γ) × (d : ⟦ C ⟧T γ) × (⟦ δ ⟧Th a ≡ d) × ⟦ x ⟧V γ d
```

An entry is a quadruple (a, d, q, v): the earlier entries a, a point d of the telescope C of the family x, a path q from the selection $[\![\delta]\!]\, a$ to d, and a value v of the family at d. By contracting the singleton (d, q) this is equivalent to the naive $\Sigma\,(a : [\![A]\!]\,\gamma)\,.\,[\![x]\!]\,\gamma\,([\![\delta]\!]\,a)$. The advantage is that a kept entry no longer has to transport its value; it only extends the stored path:

```
⟦ t-k σ ex e ⟧Th (a , d , q , v) = ⟦ σ ⟧Th a , d , kp δ σ δ' e a • q , v
  where kp δ σ δ' e a = sym (⟦∘⟧ δ σ a) • ≈ e a
```

(We have suppressed the code `ex : CodeV x y` relating the source and target variables; in the formalisation it is decoded to a path in the set of variables and matched with `refl`.) A thinning never changes the point d or the value v of an entry. Consequently, a path between the interpretations of two thinnings has no component in the values: it is a path in the earlier entries together with a 2-path between stored paths.

Codes act as paths. The second change concerns the stored codes. We need a path from $[\![\delta \circ \sigma]\!]\, a$ to $[\![\delta']\!]\, a$ for every code `e : Code (δ ∘ σ) δ'`. Rather than applying the interpretation to the decoded equation `Code-sound e`, which does not compute, we give codes a semantic action, by recursion:

```
≈ : Code σ τ → (a : ⟦ A ⟧T γ) → ⟦ σ ⟧Th a ≡ ⟦ τ ⟧Th a
```

On `tε` and `t-d` it is `refl` and the recursive call. On two keeps it is a path in the Forded Σ-type, whose components are $\approx$ on the kept thinnings and a 2-path between stored paths, described below.

Weakening. The variables `zero : Var (Γ ▷ B) (wkTy B p)` and `suc x : Var (Γ ▷ B) (wkTy A p)` live over weakened telescopes. A telescope `A` of `Γ`, moved into `Γ ▷ B`, has the same shape, but its entries refer to the families `suc x` instead of `x`. Its interpretation `⟦ wkTy A p ⟧T (γ , X)` is therefore not definitionally `⟦ A ⟧T γ`: the two agree only if the interpretations of `wkThin δ p` and `δ` agree, and that holds only up to a path. So we do not identify them but relate them by a map, defined by recursion on `A`,

```
unwk : ⟦ wkTy A p ⟧T (γ , X) → ⟦ A ⟧T γ
unwk ε b                    = tt
unwk (ext A x δ) (a , d , q , v)
  = unwk A a , unwk C d , sym (unwk-nat δ a) • cong (unwk C) q , v
```

together with its naturality for thinnings,

```
unwk-nat : unwk C (⟦ wkThin δ p ⟧Th a) ≡ ⟦ δ ⟧Th (unwk A a)
```

The variables are interpreted through it,

```
⟦ zero  ⟧V (γ , X) b = ⌞ X (unwk B b) ⌟
⟦ suc x ⟧V (γ , X) b = ⟦ x ⟧V γ (unwk A b)
```

which is what makes the value `v` in the definition of `unwk` well typed. Again the Forded semantics pays off: `unwk` leaves the value alone and only corrects the stored path, using `unwk-nat`.

Paths of kept entries. All paths between kept entries have the same form. We state it for a general Forded Σ-type. Let `B` be a type of earlier entries, `D` a type of points with a selection `f : B → D`, and `V` a family over `D`; for ⟦ `ext A x δ` ⟧`T γ` these are ⟦ `A` ⟧`T γ`, ⟦ `C` ⟧`T γ`, ⟦ `δ` ⟧`Th` and ⟦ `x` ⟧`V γ`. Given earlier entries `b b' : B`, a point `d : D`, stored paths `q : f b ≡ d` and `q' : f b' ≡ d`, and a value `v : V d`, we can form the two entries

```
(b , d , q , v) , (b' , d , q' , v) : (b : B) × (d : D) × (f b ≡ d) × V d
```

which share their point and their value. Given moreover a path `r : b ≡ b'` between the earlier entries and a 2-path `s : sym (cong f r) • q ≡ q'`, saying that going back along the image of `r` and then along `q` gives `q'`, we have

```
pathK r s : (b , d , q , v) ≡ (b' , d , q' , v)
```

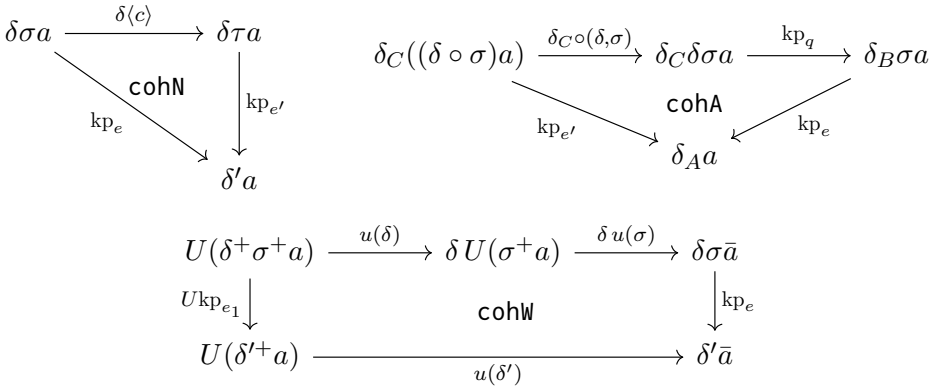
defined by J on `r`.

`pathK` organises everything that follows. It splits a path between kept entries into a *base* part `r`, which will be given by recursion or induction on the thinnings, and a *stored-path* part `s`, which is where the coherences come in. One level up, two composites of `pathK`s are equal if their base paths are equal and their stored-path parts agree; the latter is an equation between 2-paths, and this is the only place where we will use that the semantic types are groupoids (Section 5.3).

The interpretation is defined by mutual recursion: the interpretation of thinnings ⟦`_`⟧`Th`, the action of codes `≈`, the composition path ⟦`∘`⟧, and `unwk` and `unwk-nat`. In each of `≈`, ⟦`∘`⟧ and `unwk-nat`, the case of a kept entry is a `pathK`, whose base path is a recursive call and whose second component is one of three 2-cells:

cell	used in	statement
`cohN`	`≈` on keeps	stored paths are natural in codes
`cohA`	⟦`∘`⟧ on two keeps	stored paths of two keeps compose to that of the composite
`cohW`	`unwk-nat` on keeps	weakening commutes with stored paths

Their types say that certain diagrams of paths commute. We omit $[\![-]\!]$, writing $\delta\sigma a$ for $[\![\delta]\!]([\![\sigma]\!]\, a)$, write $\langle c\rangle$ for the code path `≈ c a`, and write kp_e for the stored path `kp δ σ δ' e a` of a keep with code `e : Code (δ ∘ σ) δ'`.



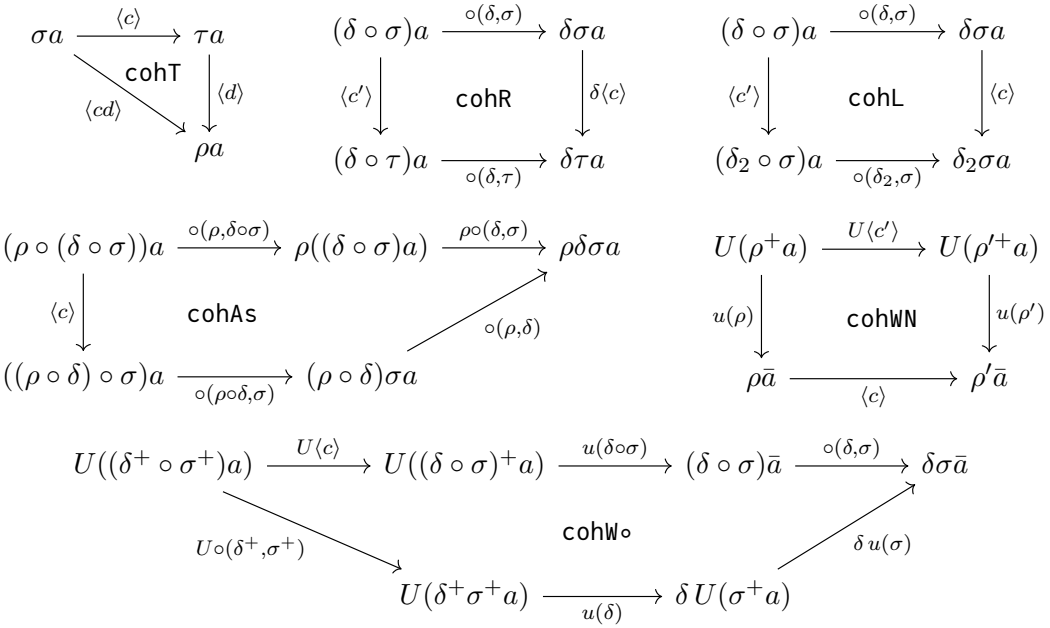
- `cohN`: for `c : Code σ τ`, moving along the code path of the kept thinnings and then using the stored path of the target keep is the stored path of the source keep. So stored paths are natural in codes.

- `cohA`: for keeps `δ` (with code `q`) and `σ` (with code `e`), the two stored paths compose to the stored path of the composite keep (with code `e'`), up to the composition path.
- `cohW`: here U = unwk, δ^+ = `wkThin δ p`, $\bar{a} = \mathrm{unwk}\, a$, and $u(\delta)$ is `unwk-nat δ`. The image under U of the stored path of a weakened keep is the stored path at $\bar{a}$, up to the weakening comparisons.

This is where the naive interpretation used UIP: it needed two transports to agree, and now we need two paths to be equal, but only the specific ones that arise.

5.2 The 2-cells

The three stored-path cells are not proved directly. They are pasted from a small set of *generic* 2-cells, which are proved by induction on the syntax. There are six generic 2-cells. We write $\circ(\delta, \sigma)$ for `⟦∘⟧ δ σ a`, and $u(\delta)$, U, δ^+, $\bar{a}$ and $\langle c \rangle$ as before; each cell says that its diagram commutes.



Here $c, c', cd, \ldots$ are codes between the relevant thinnings. In words: code paths compose (`cohT`), composition is natural in codes on the right and on the left (`cohR`, `cohL`), composition is associative up to a code (`cohAs`), and the weakening comparison is natural in codes and compatible with composition (`cohWN`, `cohW∘`). So the semantics has the structure of a pseudofunctor on the category of thinnings: `≈` is its action on codes, `⟦∘⟧` its compositor, `unwk-nat` a pseudonatural comparison, and the generic 2-cells are the corresponding naturality and associativity data.

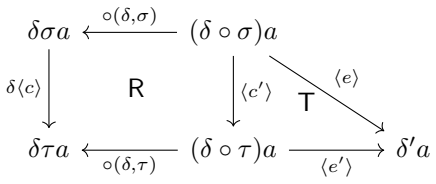
The stored-path cells as pastings. Each stored-path cell is a pasting of generic 2-cells (Figure 1). The pasting is done by a lemma of pure path algebra, proved by J, since it holds for arbitrary paths. For `cohN` it is

```
lemN : m • n ≡ k • l → k • e₂ ≡ e₁ → sym n • (sym m • e₁) ≡ sym l • e₂
```

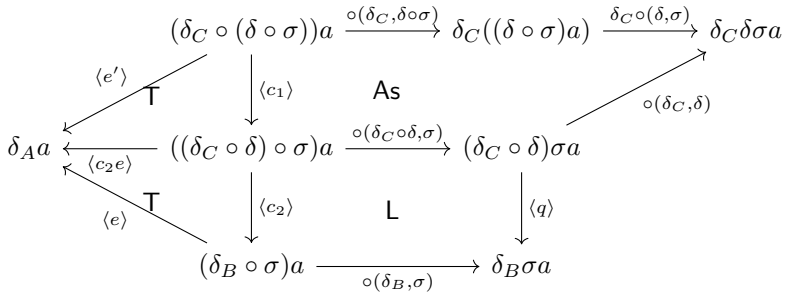
instantiated with $m = \circ(\delta, \sigma)$, $n = \delta\langle c \rangle$, $k = \langle c' \rangle$, $l = \circ(\delta, \tau)$, $e_1 = \langle e \rangle$ and $e_2 = \langle e' \rangle$. Then the first hypothesis is the `cohR` square for c and the induced code c', the second is the `cohT` triangle

for c', e' and e, and since the stored path is (writing, as in Agda, $p \bullet q$ for path composition) $\mathrm{kp}_e = \circ(\delta,\sigma)^{-1} \bullet \langle e \rangle$, the conclusion $\delta\langle c\rangle^{-1} \bullet \mathrm{kp}_e = \mathrm{kp}_{e'}$ is `cohN`: a square and a triangle pasted along $\langle c' \rangle$ (Figure 1(a)). In the same way `lemA` pastes `cohAs`, `cohL` and two instances of `cohT` into `cohA`, and `lemW` pastes `cohW∘`, `cohWN` and `cohT` into `cohW`.

(a) `cohN`



(b) `cohA`



(c) `cohW`

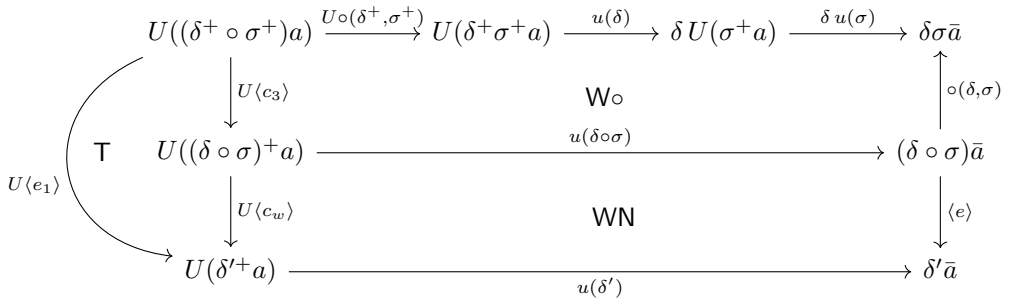


Figure 1: The stored-path cells as pastings of the generic 2-cells. We omit $[\![-]\!]$: $\delta\sigma a$ stands for $[\![\delta]\!]([\![\sigma]\!]\, a)$ and $(\delta \circ \sigma)a$ for $[\![\delta \circ \sigma]\!]\, a$, and, as in Agda, $p \bullet q$ for path composition. A stored path is $\mathrm{kp}\,\delta\,\sigma\,\delta'\,e = \circ(\delta,\sigma)^{-1} \bullet \langle e \rangle$. In each diagram the cell is the equality of the two paths around the outer boundary, from $\delta\sigma a$ in (a) and from the top right corner in (b) and (c), and it is obtained by pasting the labelled regions; this is what the J-lemmas `lemN`, `lemA`, `lemW` do. (a) $\mathrm{kp}\,\delta\,\sigma\,\delta'\,e = \delta\langle c\rangle \bullet \mathrm{kp}\,\delta\,\tau\,\delta'\,e'$, where c' is the code induced by c. (b) $\delta_C \circ (\delta,\sigma)^{-1} \bullet \mathrm{kp}\,\delta_C\,(\delta\circ\sigma)\,\delta_A\,e' = \mathrm{kp}\,\delta_C\,\delta\,\delta_B\,q \bullet \mathrm{kp}\,\delta_B\,\sigma\,\delta_A\,e$, where c_1 is the associativity code, c_2 the code induced by q, and $c_2 e$ their composite with e. (c) $U = \mathrm{unwk}$, $\delta^+ = \mathrm{wkThin}\,\delta\,p$ and $\bar{a} = \mathrm{unwk}\,a$: the image under U of the stored path of the weakened keep agrees with the stored path at $\bar{a}$, up to the weakening comparisons u.

Proving the generic 2-cells. The generic 2-cells are proved by induction on the thinnings, mutually with the interpretation. On `tε` and `t-d` they reduce to the induction hypothesis or hold by `refl`. On keeps both sides are composites of `pathK`s, and two such composites are equal if

their base paths are equal (the induction hypothesis) and their stored components agree. The stored components are 2-paths between paths in ⟦ C ⟧T γ, so their agreement is a *3-path* in ⟦ C ⟧T γ.

5.3 Where the groupoids are used

This is the only place where we use that the semantic types are groupoids. In a groupoid the path types are sets, so any two parallel 2-paths are equal and the required 3-paths exist:

```
grpdEq : isGroupoid D → {u v : D}{q q' : u ≡ v}(s s' : q ≡ q') → s ≡ s'
```

This is used in the two lemmas that compare composites of pathKs (pathK-•• and pathK-•≡), and nowhere else. So the tower of coherences stops after one step:

level	cells	proved by
1	≈, ⟦∘⟧, unwk-nat	recursion; stored parts cohN, cohA, cohW
2	cohT, cohR, cohL, cohAs, cohWN, cohW∘	induction; stored parts 3-paths
3	3-paths in the semantic types	trivial in groupoids

The naive interpretation needed the analogous statement one level lower, that 2-paths in the semantic types are trivial, which is UIP. The Forded semantics and the action of codes shift the use of truncation up by one dimension.

The formalisation. experiments/sem1/Sem1.agda has about 870 lines, contains no postulates and no holes, and uses no K (it is checked with --cubical, which implies --without-K). It uses 15 TERMINATING pragmas: the mutual recursion is structural, but it goes through the decoded codes and through the mutually defined functions in a way that Agda's termination checker does not recognise. Type checking takes about two and a half minutes. Most of the work is in the keep cases, where the J-lemmas must be applied with explicit motives, and in the cases of cohAs and cohW∘ that combine several keeps.

6 Semi-simplicial groupoids

Interpreting the uniform construction of Section 3 gives

```
⟦ SST n ⟧C : Set₁
```

for a variable n: a type of semi-simplicial groupoids up to level n, in Forded form. Each level is a family of groupoids over the Forded telescope of the boundary of a simplex, whose entries store paths from the selected faces to their points.

The file experiments/sem1/SemSST.agda checks that the construction selects the right faces. For $n = 1, 2$ there are maps from the standard, unforded semi-simplicial groupoids, written out by hand, into $[\![\mathrm{SST}\, n]\!]$, which transport each value along its stored path to the family applied to the faces it should select. These maps typecheck only if every face map of the construction selects the right lower simplices. For $n = 3$ the same check is too expensive, since Agda would have to normalise stored paths that contain the coherence proofs. But since a thinning never changes the point d or the value v of an entry, it is enough to check that each triangle face of a tetrahedron selects entries with the right points and values. These checks hold by refl, and a deliberately wrong face fails, as expected.

We have not formalised that $[\![\mathrm{SST}\, n]\!]$ is equivalent to the type of semi-simplicial groupoids up to level n written out by hand, for each numeral n. Level by level this is the contraction of the singletons (d, q), but the equivalence has to be compatible with the face maps, and we have only checked this for $n \leq 3$ as above.

The natural statement of correctness uses two-level type theory [3, 4], where semi-simplicial types up to level n can be defined for an outer natural number n. Restricting that definition to families of groupoids gives a type $\mathrm{SST}_n^{\mathrm{Grpd}}$ of semi-simplicial groupoids, and we expect

$$[\![\mathrm{SST}\, n]\!] \simeq \mathrm{SST}_n^{\mathrm{Grpd}}$$

for every outer n. Since our construction of $\mathrm{SST}\, n$ follows the recursion of [3, Section 3], the proof would go by induction along it. The skeleta are equivalent by contracting the singletons (d, q), and the main work is to show that these equivalences commute with the face maps, coherently with α: the face maps compose strictly in two-level type theory, and only up to the 2-cells of Section 5 in our semantics. We leave this for future work.

7 The next level

The same construction, with groupoids replaced by 2-types, is in `experiments/sem1/Sem2.agda`. Now the stored components of the generic 2-cells are 3-paths, which are no longer trivial. Every place where `Sem1` used `grpdEq` becomes an obligation: a 3-path between two specific pastings of 2-cells, transported along a 2-cell at smaller thinnings. There are six such obligations, one for each generic 2-cell, and unfolding the stored-path cells turns each into an equation between two pastings of the generic 2-cells, that is, a 3-cell. We expect twelve generic 3-cells:

3-cell	statement
`T3`	`cohT` is associative
`RT`, `LT`	`cohR`, `cohL` are functorial in codes
`RL`	interchange of `cohR` and `cohL`
`AsR`, `AsM`, `AsL`	`cohAs` is natural in each argument
`Pent`	the pentagon for `cohAs`
`WNT`	`cohWN` is functorial in codes
`W∘R`, `W∘L`	`cohW∘` is natural in each argument
`W∘As`	`cohW∘` is compatible with `cohAs`

These are the pseudofunctor axioms one level up. So far we have discharged the obligation for `cohT` from `T3` and `RT`, and proved `T3`. On keeps the proof of `T3` again splits into a base 3-cell, the induction hypothesis, and a stored component, which is a 4-path in a 2-type and therefore trivial. This confirms the pattern: at each level the cells are equations between two given parallel pastings of the cells below, so no boundary data is ever needed, and the keep cases are discharged by the induction hypothesis together with the truncation one dimension up. The remaining obligations and 3-cells are work in progress.

The pattern. The cells are indexed by simplices of the nerve of the category of thinnings. The compositor $\circ(\delta, \sigma)$ belongs to a 2-simplex with edges σ, δ, $\delta \circ \sigma$; the associator `cohAs` to a 3-simplex, whose four faces are compositor cells; the pentagon to a 4-simplex, with five associator faces. The code cells express that the semantics respects the equations of the syntax, and the

weakening cells that it commutes with context extension. To define the interpretation into k-types uniformly in an internal $k : \mathbb{N}$, one would have to describe this whole tower as a single statement, "parallel pastings of the generators agree", generated by induction on the dimension. Stating it needs a uniform notion of pasting, and this is where higher categories come back: the coherences say that the interpretation is a k-coherent functor from the thinnings into the universe of k-types, and the latter is described by semi-simplicial types of higher dimension.

8 Discussion

What has been achieved. The obstruction of [2] is not a limitation of the syntax, nor of the construction of $\mathrm{SST}\, n$ in it. It is the coherence of the semantic composition, which the naive interpretation takes from UIP. With a Forded semantics of extension and a semantic action of codes, these coherences can be proved by induction, and for groupoids the induction closes after one level. For 2-types it needs one more level, and the shape of that level is now known.

Induction-recursion. The construction relies on the inductive-recursive syntax: composition and weakening are defined by recursion and occur in the indices of constructors, so that the syntax has no transports and its recursive equations hold definitionally. There are general accounts of induction-recursion [5, 6] and of induction-induction [9], but we know of none that covers this form. In [2] we also considered syntaxes without induction-recursion: a Forded higher inductive-inductive type, which is a set but whose naive model needs UIP, and which could instead be interpreted through normal forms, that is, through the inductive-recursive syntax. A different possibility is a Segal-style syntax in which composition is a horn filler with a contraction, and the thinnings are truncated at the level of the semantics (file `experiments/segal/TT1.agda`, a higher inductive-inductive type at the level of groupoids). An interpretation of that syntax in groupoids would avoid induction-recursion altogether, and would organise the coherences as the Segal condition for the universe. We have not yet attempted it.

Higher categories. The pattern of Section 7 suggests organising the construction with higher categories. Instead of extracting the coherences of each level from the syntax, we would define, mutually and by induction on the truncation level k, semi-simplicial k-types and $(k, 1)$-categories presented by them, and use the higher-categorical structure of the universe of k-types to interpret the syntax at the next level. The coherences of Sections 5 and 7 would then be instances of this structure rather than ad hoc lemmas. We think this is feasible, and conceptually the right picture: truncated semi-simplicial types and higher categories should be definable together. Passing from the truncated versions to untruncated semi-simplicial types is a separate step. If the types $\mathrm{SST}_n^{(k)}$ of semi-simplicial k-types are definable uniformly in k, together with maps $\mathrm{SST}_n^{(k+1)} \to \mathrm{SST}_n^{(k)}$, we can simply *define*

$$\mathrm{SST}_n = \lim_k \mathrm{SST}_n^{(k)},$$

the type of Postnikov towers of truncated semi-simplicial types. The maps are not levelwise truncation: a truncated family has to live over the boundary built from the truncated lower levels, so they are the Postnikov truncation of the diagram (truncate the total spaces of simplices, then take fibres over the new boundaries), and defining them uniformly in n needs the semantics to be natural in the truncation level. Whether this limit is *the* type of semi-simplicial types depends on whether types are determined by their truncations (Postnikov towers converging). This is where hypercompleteness comes in (Whitehead's principle: ∞-connected maps are equivalences), which

is not provable in book HoTT. So the outcome would be that semi-simplicial types are definable up to Postnikov completion, and agree with the intended ones in models where Postnikov towers converge, such as simplicial sets. It would also suggest why a direct definition of the untruncated type is hard to find: it would have to work in models where towers do not converge.

Use of AI tools

This note and the accompanying Agda development were produced in collaboration with Claude (Anthropic), used through Claude Code. Claude drafted text and Agda code, carried out experiments in Agda, and searched the literature, under the author's direction. The author has checked the content and is responsible for it.

References

[1] Henry Ford, in collaboration with Samuel Crowther. *My Life and Work*. Doubleday, Page & Company, 1922.

[2] Thorsten Altenkirch. Any colour you like, as long as it's UIP: semi-simplicial types from weakenings and thinnings. arXiv:2610.29585 [cs.LO], 2026. doi:10.48550/arXiv.2610.29585.

[3] Thorsten Altenkirch, Paolo Capriotti and Nicolai Kraus. Extending homotopy type theory with strict equality. In *CSL 2016*, LIPIcs 62, pages 21:1–21:17, 2016.

[4] Danil Annenkov, Paolo Capriotti, Nicolai Kraus and Christian Sattler. Two-level type theory and applications. *Mathematical Structures in Computer Science*, 33(8):688–743, 2023.

[5] Peter Dybjer. A general formulation of simultaneous inductive-recursive definitions in type theory. *Journal of Symbolic Logic*, 65(2):525–549, 2000.

[6] Peter Dybjer and Anton Setzer. A finite axiomatization of inductive-recursive definitions. In *TLCA 1999*, LNCS 1581, pages 129–146. Springer, 1999.

[7] The Univalent Foundations Program. *Homotopy Type Theory: Univalent Foundations of Mathematics*. Institute for Advanced Study, 2013.

[8] Astra Kolomatskaia and Michael Shulman. Displayed type theory and semi-simplicial types. *Mathematical Structures in Computer Science*, published online 2025. arXiv:2311.18781.

[9] Fredrik Nordvall Forsberg and Anton Setzer. Inductive-inductive definitions. In *CSL 2010*, LNCS 6247, pages 454–468. Springer, 2010.

[10] Michael Shulman. Homotopy type theory should eat itself (but so far, it's too big to swallow). Blog post, https://homotopytypetheory.org/2014/03/03/hott-should-eat-itself/, 2014.

[11] Univalent Foundations special year, IAS. Semi-simplicial types (open problem page). https://ncatlab.org/ufias2012/show/Semi-simplicial+types, 2012–13.

[12] Andrea Vezzosi, Anders Mörtberg and Andreas Abel. Cubical Agda: a dependently typed programming language with univalence and higher inductive types. *Journal of Functional Programming*, 31:e8, 2021.